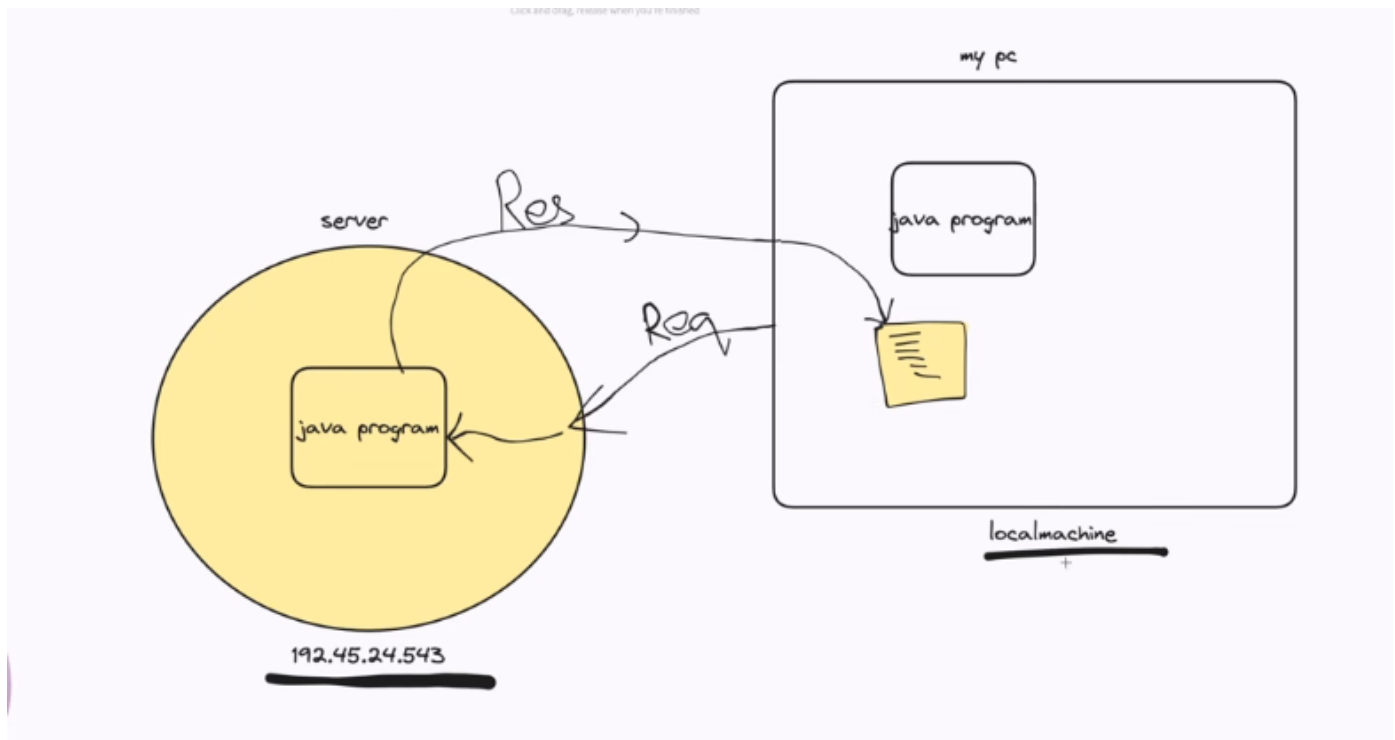


Servlet

Java programs that run on server, can be accessed from anywhere by accessing the server ip.

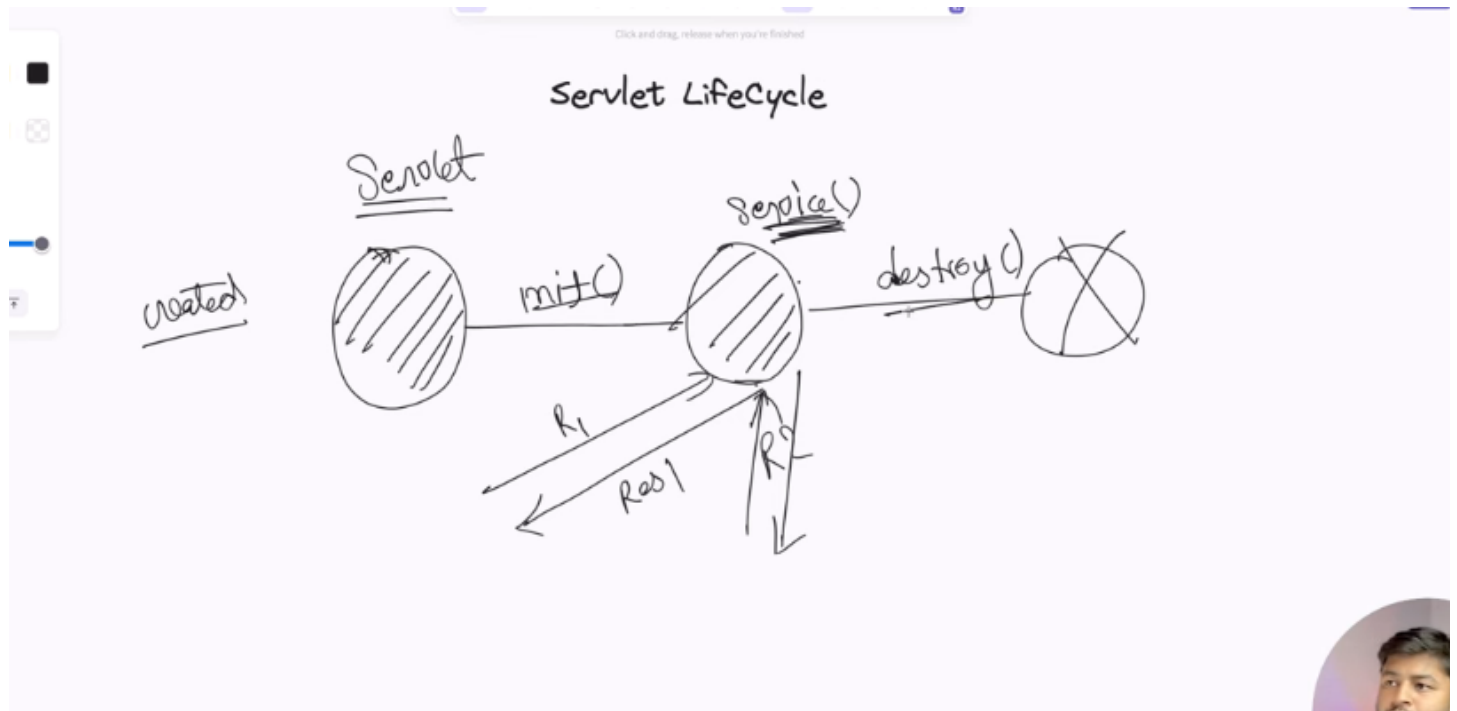


Servlet LifeCycle :

Managed by Servlet Container or Web Container (Mainly the Server like tomcat).

The lifecycle of a servlet is controlled by the **container** (e.g., Tomcat), and it goes through the following stages:

1. **Loading and Instantiation:** The servlet class is loaded and an instance is created.
2. **Initialization (`init` method):** The `init` method is called once to initialize the servlet.
3. **Request Handling (`service` method):** The `service` method is called for each request to process it.
4. **Destruction (`destroy` method):** The `destroy` method is called once before the servlet is removed from service.



To use Servlet Interface and Various related class u have to include **jakarta.servlet** dependency in your pom.xml

You can add any html , css or js file in Web-App folder. And we use JSP file because unlike html here we can write java code as well for Example :

To use Java Code or print anystring in jsp , we use

<%= %>

To use css include this in your <head tag

<link rel="stylesheet" href=" <%= application.getContextPath() %>/css/styles.css"/>

And to use Js use

<script src="<%= application.getContextPath() %>/js/script.js"> </script>

run

configuration

For Creating Servlet there are three methods

- 1) **Servlet Interface**
- 2) **GenericServlet** Abstract Class
- 3) **HttpServlet** Abstract Class (Most Preff)

Since Servlets are used to perform actions based on user/brower's incoming http request and these requests are associated with urls therefore , every

servlet should be mapped to a url

And mapping can be done in two ways :

- 1) **Annotation Based** `@WebServlet("/url")`
- 2) **Web.xml**
(refer to the code in 8_Servlet)

Tomcat Related

To check if TOMcat is running or not , just type in your browser

<https://localhost:portno>

tomcat/bin/startup file
shutdown file

Deploy war

file on Tomcat

- 1) Start Tomcat
- 2)
<https://localhost:portno>
- 3) Go to manage Apps
- 4) Upload and deploy war file

Terminologies related to Servlet and Web:

Content Type: what type of req / resp. you are sending with http header.

There are many content types. The

- text/html
- text/plain
- application/msword
- application/vnd.ms-excel
- application/jar
- application/pdf
- application/octet-stream
- application/x-zip
- images/jpeg
- images/png
- images/gif
- audio/mp3
- video/mp4
- video/quicktime etc.

2) [HttpServlet Methods](#) :

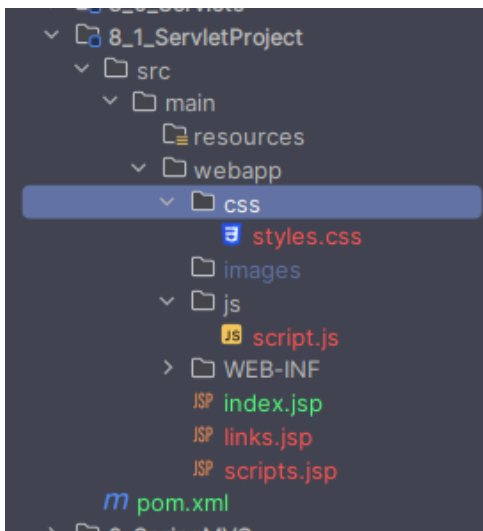
Summary of Main Methods:

- **doGet:** For retrieving data (GET requests).
- **doPost:** For submitting data (POST requests).
- **doPut:** For updating data (PUT requests).
- **doDelete:** For deleting data (DELETE requests).
- **doOptions:** For checking allowed methods (OPTIONS requests).
- **doTrace:** For diagnostic purposes (TRACE requests).

when u want some information from server we use doGet , the information is visible on the url / but when u send some info to server we use doPost or for modifying we use doPut method . doGet is idempotent means , running it as many times as you want will not affect/modify server or database unlike doPost/doPut methods

To print something in response we use
`resp.getWriter().print()`

Servlet Feedback Form



`<%@include file="" %>`

Most Important Thing , how to connect form with servlet

```
<form action="<%=application.getContextPath() %>/feedback" method="post"
```

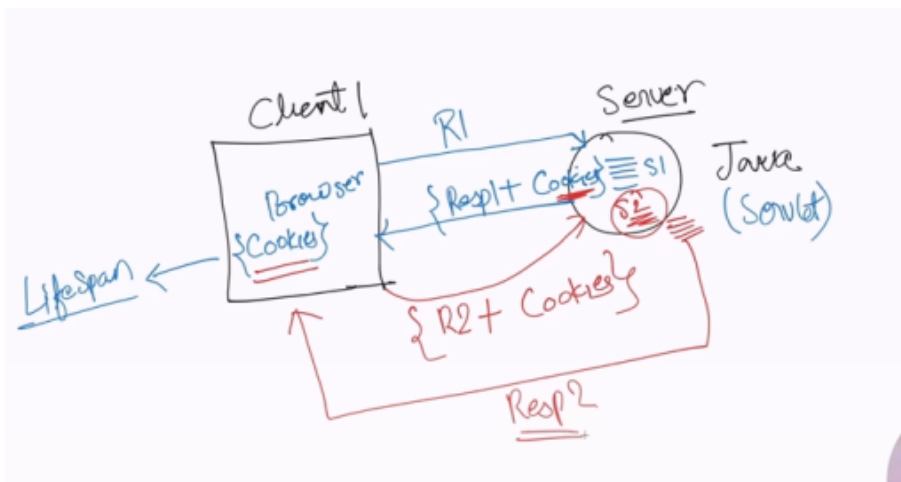
```
@WebServlet("/feedback")
```

Session Tracking Techniques

There are four techniques used in Session tracking:

1. **Cookies**
2. **Hidden Form Field**
3. **URL Rewriting**
4. **HttpSession**

1) Cookies



To Check Current Cookies for a website ,
Inspect -> Application -> Cookies
You can also manually add cookies by right click

How to **Create and Send** Cookie ?

```
java

Cookie cookie = new Cookie("username", "John");
cookie.setMaxAge(60 * 60 * 24); // Cookie will expire in
response.addCookie(cookie);      // Add the cookie to th
```

How to **retrieve** Cookie?

```
java
Copy code

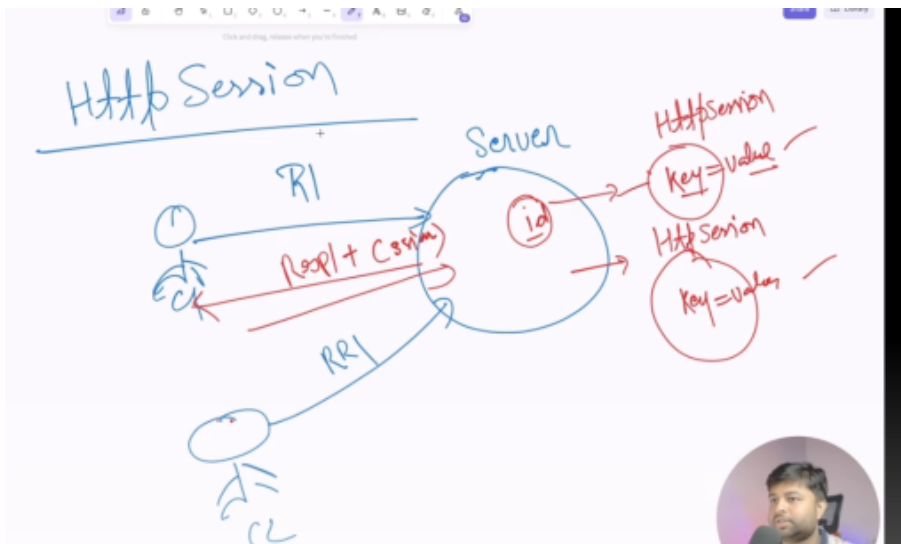
Cookie[] cookies = request.getCookies(); // Retrieve all cookies from the request
if (cookies != null) {
    for (Cookie cookie : cookies) {
        if ("username".equals(cookie.getName())) {
            String username = cookie.getValue(); // Get the cookie's value
            // Do something with the username
        }
    }
}
```

How to **delete** a Cookie?

```
java

Cookie cookie = new Cookie("username", "");
cookie.setMaxAge(0); // Cookie is immediately deleted
response.addCookie(cookie);
```

2) HTTPSession



When the user first Sends a request to the server , the server creates an HttpSession Object having a unique SessionId , now this SessionID is sent back to the browser(client) in the form of a cookie example(**JSESSIONID 87687989098**), now if the browser req again, the server checks if the sessionID already exists or not.

The main difference between a cookie and HttpSession is that , cookie is stored in browser whereas the HTTPSessions as many attributes (key-value pairs) as u want on the server side, and only SessionID is passed to the client(Browser) .

Create HttpSession

```
String user = "Rjt";
String pass = "cdrRjt@123";

HttpSession session = req.getSession();
session.setAttribute( s: "user", user);
session.setAttribute( s: "pass", pass);
session.setMaxInactiveInterval(2*60);

PrintWriter writer = resp.getWriter();
resp.setContentType("text/html");

writer.println("<h1> Cookie Successfully Set </h1> ");
}
```

Get HttpSession

```
HttpSession session = req.getSession( b: false);

String user = (String)session.getAttribute( s: "user");
String pass = (String) session.getAttribute( s: "pass");

System.out.println("The user logged is : " + user + " Having password + " + pass);
writer.println("
    <h1> Successfully Logg in </h1>
    ");
```

Servlet Config / Servlet Context :

-

Key Differences:

Feature	ServletConfig	ServletContext
Scope	Per servlet	Shared across the entire application
Usage	Provides initialization parameters to a single servlet	Shares data and settings across all servlets
Initialization	Servlet-specific config (e.g., API key for a servlet)	Application-wide config (e.g., app version)

Summary:

- **ServletConfig:** Used to pass **initialization parameters** to a specific servlet.
- **ServletContext:** Used to share **data or settings** across the entire application, accessible by all servlets.